14. Write a code using RAW sockets to implement packet sniffing.

Aim

To write a Python program using RAW sockets to capture and examine network packets. To understand how packets are received at the network layer, study the basic structure of IP packets, and observe the information contained in network traffic.

Procedure

1. Open Visual Studio Code and create a new folder named CN_Experiment_14. Inside the folder, create a Python file named packet_sniffer.py.
2. Ensure that Python is installed on the system. Open the VS Code terminal and verify the installation by executing python --version. Since RAW sockets require special permissions on many operating systems, run the program with the required administrative or root privileges.
3. Import the Python socket module, which provides the functions required to create and configure network sockets. The program uses a RAW socket to receive network packets rather than communicating only through a standard TCP or UDP connection.
4. Obtain the local machine's IP address using the socket functions. This address is used to bind the socket to the local network interface or address, depending on the operating system and the selected implementation.
5. Create a RAW socket using the socket.socket() function with the appropriate address family and protocol. For example, socket.AF_INET specifies IPv4, while socket.SOCK_RAW requests a RAW socket. The protocol parameter determines which type of packets the socket receives.
6. Bind the socket to the appropriate local IP address and port configuration supported by the operating system. Unlike a normal application socket, a RAW socket can provide access to lower-level packet information, depending on the operating system, protocol, and permissions.
7. Configure any required socket options supported by the selected platform. On systems that support it, packet-capture settings may be required to receive traffic from a network interface. The exact configuration differs between Windows and Linux.
8. Use a loop to receive packets continuously. The recvfrom() function can be used to receive packet data and associated address information. Store the received data in a variable so that the program can process the packet.
9. Examine the received packet data. For IPv4 packets, the IP header contains information such as the source IP address, destination IP address, protocol number,

packet length, and time-to-live (TTL). These fields help identify where a packet originated and where it is intended to go.

10. Extract the source and destination IP addresses from the IP header. Convert the relevant binary address fields into readable IP addresses using suitable functions from the socket module.
11. Display the captured packet information on the terminal. For each packet, print the source IP address, destination IP address, protocol information, and other header fields that the program successfully extracts.
12. Generate network traffic while the sniffer is running. For example, open another terminal and execute a ping command to a reachable destination, or access a website. Observe whether the sniffer receives packets associated with that traffic.
13. Repeat the experiment with different network activities and compare the captured packet information. The output can help demonstrate that network communication consists of packets containing addressing and protocol information.
14. If the program displays a permission error, check whether the terminal has the required privileges and whether the operating system permits the selected RAW socket operation. On Windows, some packet-capture methods and socket options differ from those available on Linux. If the required operation is unsupported or blocked, an appropriate packet-capture method for that platform must be used.
15. After completing the experiment, stop the packet-capture loop using the program's termination method. Review the displayed output and identify the source and destination addresses and other packet-header details.

packet_sniffer.py

```
import socket
HOST = socket.gethostbyname(socket.gethostname())

sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
sniffer.bind((HOST, 0))
sniffer.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)

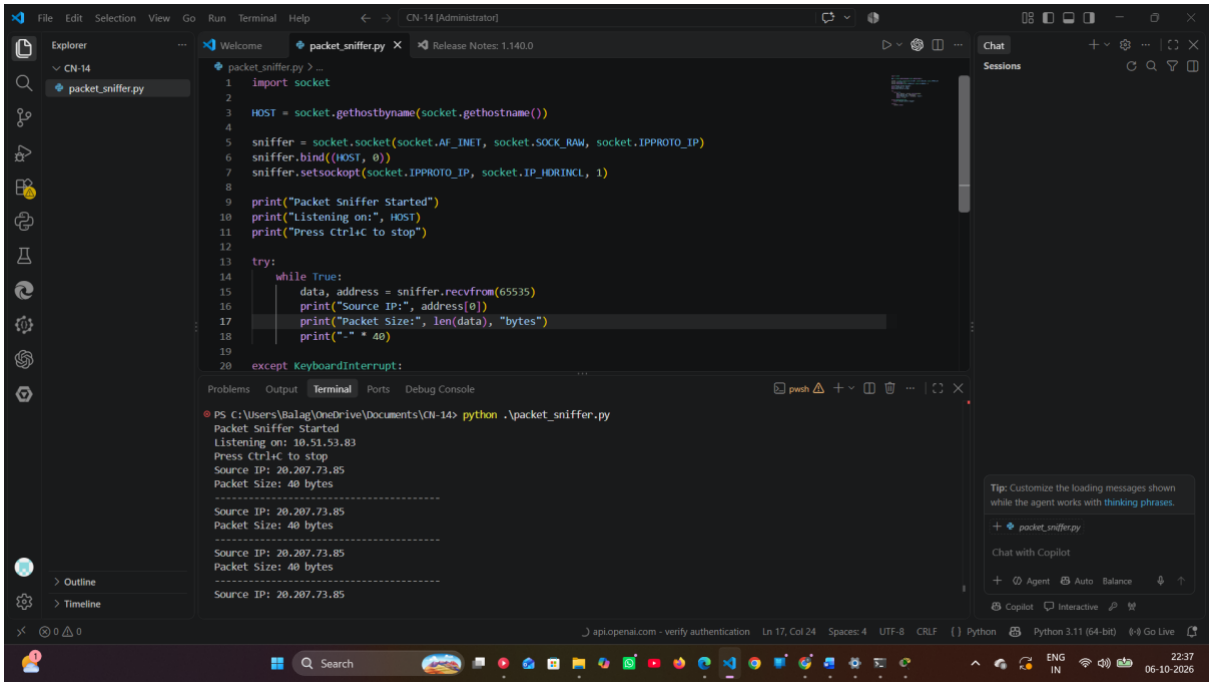
print("Packet Sniffer Started")
```

```
print("Listening on:", HOST)
print("Press Ctrl+C to stop")

try:
    while True:
        data, address = sniffer.recvfrom(65535)
        print("Source IP:", address[0])
        print("Packet Size:", len(data), "bytes")
        print("-" * 40)

except KeyboardInterrupt:
    print("\nPacket Sniffer Stopped")

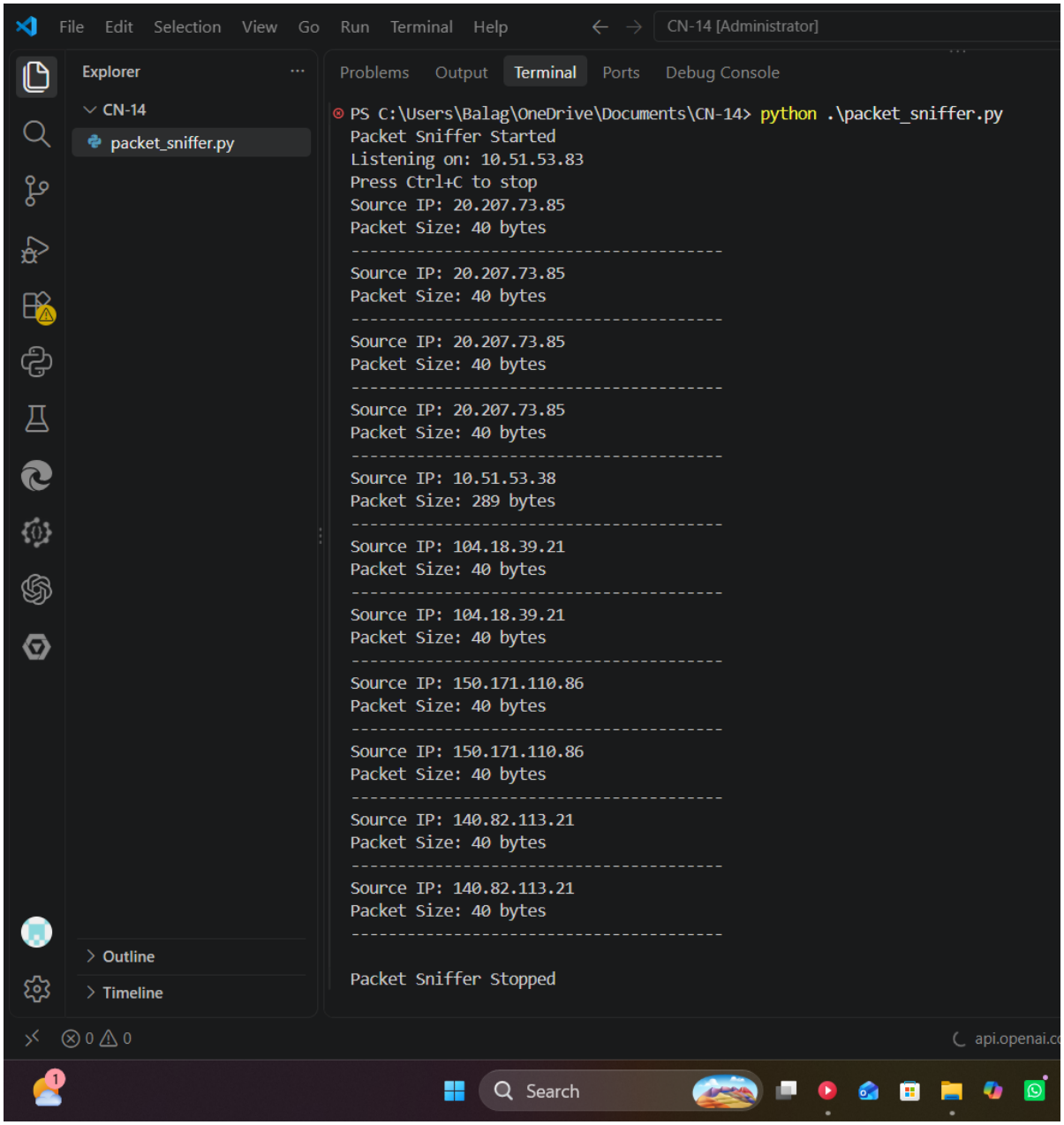
finally:
    sniffer.close()
```


File Edit Selection View Go Run Terminal Help
CN-14 [Administrator]
Explorer
CN-14
packet_sniffer.py
Welcome
packet_sniffer.py
Release Notes: 1.140.0
packet_sniffer.py > ...
1 import socket
2
3 HOST = socket.gethostbyname(socket.gethostname())
4
5 sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
6 sniffer.bind((HOST, 0))
7 sniffer.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
8
9 print("Packet Sniffer Started")
10 print("Listening on:", HOST)
11 print("Press Ctrl+C to stop")
12
13 try:
14 while True:
15 data, address = sniffer.recvfrom(65535)
16 print("Source IP:", address[0])
17 print("Packet Size:", len(data), "bytes")
18 print("-" * 40)
19
20 except KeyboardInterrupt:
Problems Output Terminal Ports Debug Console
pwsh
PS C:\Users\Balag\OneDrive\Documents\CN-14> python .\packet_sniffer.py
Packet Sniffer Started
Listening on: 10.51.53.83
Press Ctrl+C to stop
Source IP: 20.207.73.85
Packet Size: 40 bytes

Source IP: 20.207.73.85
Packet Size: 40 bytes

Source IP: 20.207.73.85
Packet Size: 40 bytes

Source IP: 20.207.73.85
Chat
Sessions
Tip: Customize the loading messages shown while the agent works with thinking phrases.
packet_sniffer.py
Chat with Copilot
Agent Auto Balance
Copilot Interactive
Outline
Timeline
api.openai.com - verify authentication
Ln 17, Col 24
Spaces: 4
UTF-8
CRLF
Python
Python 3.11 (64-bit)
Go Live
Search
ENG
IN
22:37
06-10-2026



Result

Thus, a basic packet-sniffing program using Python RAW sockets was studied and implemented. The experiment demonstrated how packet data can be received and examined to identify network-layer information, including source and destination IP addresses and protocol details. It also highlighted the importance of operating-system support and appropriate permissions when capturing network packets.